

Fiche récapitulative – STA211

Complément `fr_Tanagra_Tensorflow_Keras_R`
TensorFlow / Keras sous R – Perceptron simple et multicouche

Tanagra Data Mining – R. Rakotomalala (avril 2018)

14 juin 2026

Résumé

Ce tutoriel présente l'utilisation des bibliothèques TensorFlow / Keras sous R, en suivant les mêmes étapes que le document équivalent sous Python. La transposition $R \leftrightarrow Python$ est particulièrement simple.

Table des matières

1	Installation sous R	2
2	Données	2
3	Perceptron simple	2
4	Perceptron multicouche	3
5	Comparaison R vs Python	3
	Questions clés (QCM)	3

Installation sous R

```
install.packages("keras")
library(keras)
install_keras() # installe Keras + TensorFlow (backend)
```

`install_keras()` installe à la fois le cœur de Keras et le moteur TensorFlow. Keras n'est qu'un *front-end* : c'est TensorFlow qui effectue les calculs.

Données

Problème de discrimination binaire dans le plan. 2000 observations, 2 variables prédictives X_1, X_2 . Frontière parabolique :

$$\text{Si } 0.1 \times X_2 > X_1^2 \implies \text{pos, sinon neg}$$

Préparation :

```
D <- read.table("artificial2d_data2.txt", sep="\t", header=T, dec=".")
y <- ifelse(D$Y=="pos", 1, 0)
set.seed(100)
idTrain <- sample(1:nrow(D), 1500, replace=F)
XTrain <- as.matrix(D[idTrain, 1:2])
yTrain <- y[idTrain]
XTest <- as.matrix(D[-idTrain, 1:2])
yTest <- y[-idTrain]
```

1500 observations en apprentissage, 500 en test.

Perceptron simple

Architecture :

```
modelSimple <- keras_model_sequential()
modelSimple %>%
  layer_dense(units=1, input_shape=c(2), activation="sigmoid")
```

- 1 neurone de sortie (codage 1/0), sigmoïde.
- 3 paramètres à estimer : a_0 (biais) $+ a_1 X_1 + a_2 X_2$.
- Biais activé par défaut (`use_bias = True`).

Compilation et apprentissage :

```
modelSimple %>% compile(
  loss="binary_crossentropy",
  optimizer="adam",
  metrics="accuracy"
)
modelSimple %>% fit(x=XTrain, y=yTrain, epochs=150, batch_size=10)
```

Poids estimés : $a_0 = -1.283$, $a_1 = 0.256$, $a_2 = 1.947$.

Performances :

- Apprentissage : `loss = 0.6429`, `accuracy = 62.2 %`
- Test (`predict_class + confusionMatrix`) : `accuracy = 67.8 %`
- Évaluation Keras (`evaluate`) : `loss = 0.6206`, `acc = 67.8 %`

Perceptron multicouche

Architecture : 1 couche cachée (3 neurones) + 1 couche de sortie (1 neurone), activation sigmoïde partout.

```
modelMc <- keras_model_sequential()
modelMc %>%
  layer_dense(units=3, input_shape=c(2), activation="sigmoid") %>%
  layer_dense(units=1, activation="sigmoid")
```

Nombre de paramètres :

- Entrée → cachée : $(2 \times 3) + (1 \times 3) = 9$
- Cachée → sortie : $(3 \times 1) + (1 \times 1) = 4$
- **Total : 13 paramètres**

Compilation et apprentissage : mêmes paramètres (binary_crossentropy, adam, 150 epochs, batch_size=10).

Performances :

- **Accuracy test : 97.4 %**
- Nette amélioration par rapport au perceptron simple (67.8 %).
- Le réseau non linéaire (couche cachée) capture mieux la frontière parabolique.
- Taux de reconnaissance sur l'échantillon test.

Comparaison R vs Python

Aspect	R	Python
Syntaxe	%>%	API fonctionnelle
Installation Keras	install_keras()	pip install keras
Fonctions	layer_dense()	layers.Dense()

À retenir. Les modes opératoires sont très similaires sous R et Python. Passer d'un langage à l'autre n'est pas un problème.

Questions clés (QCM)

1. Quelle fonction installe Keras + TensorFlow sous R ? **R. `install_keras()`.**
2. Quel opérateur chaîne les instructions Keras sous R ? **R. `%>%` (pipe).**
3. Combien de paramètres pour le perceptron simple ? **R. 3 (biais + 2 poids).**
4. Combien de paramètres pour le MLP (3 neurones cachés) ? **R. 13 (9 + 4).**
5. Quelle est l'accuracy du perceptron simple en test ? **R. 67.8 %.**
6. Quelle est l'accuracy du MLP en test ? **R. 97.4 %.**
7. Quelle fonction d'activation est utilisée ? **R. Sigmoide (`sigmoid`).**
8. Quel optimiseur est utilisé ? **R. Adam.**
9. Quelle fonction de perte (loss) ? **R. `binary_crossentropy`.**
10. Quel est le nom de l'instruction de création du modèle ? **R. `keras_model_sequential()`.**